

P1860R0

C++ Networking Must Be Secure By Default

Published Proposal, 2019-09-05

This version:

<http://wg21.link/P1860R0>

Authors:

[Alex Christensen](#) (Apple)

[JF Bastien](#) (Apple)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1860R0.bs

Table of Contents

- 1 Motivation**
- 2 Performance**
- 3 Proposed API Direction**
- 4 Appendix: Security Design**
 - 4.1 Trusted Root Certificates
 - 4.2 Certificate Inspection
 - 4.3 Self Signed Certificates
 - 4.4 Accepting Invalid Certificates
 - 4.5 Key Pinning

- 4.6 Certificate Expiration
- 4.7 DNS Encryption
- 4.8 Plaintext Traffic
- 4.9 UDP and DTLS
- 4.10 Client Certificates
- 4.11 QUIC and Experimental Protocols
- 4.12 Crypto Agility
- 4.13 Public Key Sizes
- 4.14 Cipher Suites
- 4.15 SHA1 Deprecation
- 4.16 TLS1.0 Deprecation
- 4.17 Revoking Individual Certificates
- 4.18 Errors
- 4.19 Default Ports
- 4.20 Default Timeouts
- 4.21 Connection Caching
- 4.22 HTTP Caching
- 4.23 Extensions

References

Informative References

§ 1. Motivation

The C++ committee is considering adding networking to its standard library to help keep C++ competitive and modern. This will be a great addition to C++. However, we are concerned about the lack of Transport Layer Security (TLS) support in the current specification [\[N4771\]](#). We hope the Committee will choose a design that encourages developers to write software that will keep information safe, work with the internet as it is now, and continue to work with the internet as it will be for the entire lifetime of their software.

The concern we're expressing is based on the authors' joint experience implementing and deploying networking, web, internet, and security software to over a billion users.

Most users of C++ networking will want to write applications that run on a client or server and communicate across an untrusted network such as the internet. In such an environment, routers often silently [record](#) data as they pass it along, or sometimes perform more active attacks such as

manipulating the packets of information, or entirely intercepting streams and pretending to be the desired recipient. To combat these and many other types of attacks, Transport Layer Security and Public Key Infrastructure have been developed and [their use is growing quickly worldwide](#). Various groups including the [World Wide Web Consortium](#) the [Linux Foundation](#) the [Electronic Frontier Foundation](#) and [all major browser vendors](#) have been working to promote the use of secure communication. To make programs written in C++ vulnerable to these attacks would be a step in the wrong direction.

The Networking TS as it now stands provides basically the same capability as if one were using the Berkeley Sockets API. While it is true that TLS libraries can be and have been built on top of either the Networking TS or the Berkeley Sockets API, many developers (especially newly learning developers) might be ignorant of the need for TLS or its need for maintenance and would write software that is vulnerable to Man-in-the-Middle attacks or cannot interface with the growing amount of secure-only content on the internet. Using the existing design will likely prolong the use of plaintext communication. Further, they will be unable to communicate with the modern internet because many servers only support HTTPS which uses TLS. By simply running `curl -v http://apple.com/` in a terminal you can see an example of a server that requires TLS to get meaningful content. If we instead make C++ networking secure by default, naïve developers will write software that communicates securely and is able to interact with the modern web. Developers who absolutely want plaintext can still opt-out of the secure default.

We offer a sample API and implementation based on the Networking TS reference implementation in [\[P1861R0\]](#).

§ 2. Performance

A frequent pushback against security-by-default is its runtime cost, in line with C++'s "you don't pay for what you don't use". We don't think such pushback is warranted here.

Modern encryption algorithms have minimal overhead. CPU overhead for the handshake, bulk encryption and decryption vary widely depending on which of the many cipher suites are agreed upon by client and server (see [benchmark](#)), and also depending on [SIMD and special instruction support](#). Actual data transmission rates can even be faster when using TLS over some networks because costly deep packet inspection is fruitless and therefore not attempted by routers and ISPs.

TLS 1.2 requires an additional round trip for the TLS Handshake, but TLS 1.3 has introduced [0-RTT resumption](#). TCP also has a round trip required for connection establishment, but it is so commonly found useful and worth the small overhead. The same is true of TLS.

This applies to the majority of networking's uses. We therefore do not think performance concerns should prevent networking having TLS by default.

§ 3. Proposed API Direction

When constructing a TCP connection, there should be a configuration flag to specify whether TLS is enabled. We propose that TLS default to on. If it is on, you get either an encrypted TCP connection after a successful TLS handshake, or an error when either the TCP connection or TLS handshake fail. As is common, the handshake would negotiate the most secure version of TLS possible with the connected client or server. UDP should have an analog with DTLS. The specification should allow the system to maintain a set of trusted root certificates or update cipher suites and key sizes without breaking binary or source compatibility with existing programs. TLS server connections will need only certificates and private keys to be configured and then the system can do the rest, and we can encourage these keys to come from files rather than compiling keys into binaries to extend the lifetime of binaries beyond the lifetimes of the keys. TLS client connections will, by default, utilize secure options for certificate validity checking and cipher suite negotiation. If a developer wants to do advanced handshake manipulation such as adding non-standard extensions, they can implement a library on top of the plaintext TCP capability of C++ networking, but beginning developers will be able to write software that communicates securely. If a developer explicitly sets a configuration flag to specify that TLS is disabled, they will get a plaintext connection.

The existing Networking TS draft has similarities to the Berkeley Sockets API, which has been the de-facto standard interface for networking for several decades now. During those decades, the needs of secure networking have evolved considerably. The IETF has been working to define [\[IETF-INTERFACE\]](#) and [\[IETF-ARCHITECTURE\]](#) that allow for secure or insecure communication with secure being the default. We should use this work as a model when designing C++ networking and if we do so well it will meet the needs of all interested parties: those interested in communication in safe networks and those interested in communication with existing and future infrastructure in the internet.

Boost.Asio does have [a way to configure SSL](#), [as does Boost.Beast](#). This design should not be used because it requires additional configuration in order to set up a secure connection. This additional complexity barrier would promote the increase of use of insecure communication on the internet and make it harder to learn to write secure web applications.

§ 4. Appendix: Security Design

The following are some things to consider when designing Secure C++ Networking:

§ 4.1. Trusted Root Certificates

Certificate chains establish the identity of the client or server with which we are communicating. Certificate authorities verify identities with signed certificates that include references to a root certificate (and possible intermediate certificates). The other party must trust that the root certificate has not been compromised for this system to work. The other party must already have a copy of the trusted root certificates from certificate authorities that they trust. Different systems may disagree on the trustworthiness of certain certificate authorities, and the set of trusted root certificates must be actively maintained. This may be the trickiest part of standardizing TLS in C++.

Root certificates can be obtained from the system's own trusted root certificate store, or places such as [Mozilla's CA certificates](#). There needs to be a way to update these if certificates are revoked.

Developers and C++ implementations will want the ability to add or remove trusted root certificates. Systems might also want the ability to do so, such as [when this happened](#). Root certificates also expire and the list is subject to additions and removals, so the system implementing secure networking must provide updates to its root certificates.

§ 4.2. Certificate Inspection

By default establishing a connection to a peer with an untrusted or invalid certificate should be an error, but we will need to have hooks for certificate inspection so a program can have the ability to accept or reject a certificate chain during a TLS handshake. This is needed for several common security techniques.

§ 4.3. Self Signed Certificates

Generating self-signed certificates is easy but often discouraged because the certificates are not trusted by CAs with widely-distributed root certificates. People do use them so it should be possible. There should be some hoops to jump through to get self-signed certificates to work. People doing local testing can't get publicly trusted certificates easily without a public IP address, so the ability to accept self signed certificates makes testing server software much easier.

§ 4.4. Accepting Invalid Certificates

A common response to invalid or untrusted certificates is to show the user a warning during the TLS handshake. For example, if a server's certificate has recently expired, a user might want to remind the administrator to renew the certificate and then manually proceed with caution.

§ 4.5. Key Pinning

With the ability to inspect certificates comes the ability to do things like only trusting a certain certificate or public key for a certain domain regardless of whether a certificate authority has verified a certificate chain. This is sometimes used to protect against compromised certificate authorities and other PKI breakdowns. Some banks use pinned certificates to provide more security, but using pinned certificates requires a separate secure channel for sending updates when certificates expire. Including the ability to inspect certificates during TLS handshakes will allow certificate pinning to be implemented.

§ 4.6. Certificate Expiration

If you're writing server software and using PKI, you're going to need to update your certificates every 6 months to 2 years. The closest thing to permanent certificates is scripts that automatically renew server certificates with a certificate authority before they expire but that behavior should probably not be standardized in the STL. Ideally the certificates would come from a file rather than being hard-coded into executables to extend the lifetime of the server software beyond 2 years.

§ 4.7. DNS Encryption

DNS over HTTPS, DNS over TLS, and others prevent eavesdropping and various other attacks and are gaining support. The ability to do DNS over HTTPS should be as easy as setting up a connection with TLS in C++ networking.

§ 4.8. Plaintext Traffic

We should have a way to do something to prevent encryption for use in trusted networks or for communication with legacy servers. The whole point of this paper is that this must not be the only option, but it has important uses.

§ 4.9. UDP and DTLS

There should be options to use UDP with and without DTLS with secure being default.

§ 4.10. Client Certificates

We ought to be able to support them if you write some extra code. Pharmaceutical companies and militaries use them for everything because the server is sending sensitive information to the client, and it must verify its identity. Common web services do not use them, so requesting client certificates should be configurable by the server program.

§ 4.11. QUIC and Experimental Protocols

There is a steady stream of experiments that improve things. We should not standardize everything, but things like QUIC have great potential. We should establish guidelines on what constitutes "significant adoption" required for further standardization. We should also design the interface to be able to support QUIC.

§ 4.12. Crypto Agility

Cryptography is a field that is affected by new developments in computational power and algorithms. Communication that was thought to be secure in years past has been shown to be insecure today, and today's secure algorithms will likely also need updating as time progresses. If C++ networking is designed such that bytes are sent and received through a black-box secure communication system, that system can be updated to keep communication secure while keeping existing programs working with modern networks.

This introduces a new aspect of binary compatibility the C++ committee should be aware of. For example, if a certain cipher suite is shown to be insecure, an update might remove it from the set of supported cipher suite to select from during a TLS handshake. If that cipher suite happened to be the only cipher suite supported by a desired communication partner, the update will break the ability to communicate with that peer. In practice, good TLS implementations offer many cipher suites to choose from and they are kept up to date in order to prevent such problems, but networking software that is widely deployed in undermaintained networks do risk such breakage.

Here are some examples of elements of a secure communication system that may need updating in addition to the routine updating of certificates:

§ 4.13. Public Key Sizes

There need to be able to be updates on what constitutes "enough bits of security" without recompiling. "RSA claims that 1024-bit keys are likely to become crackable some time between 2006 and 2010 and that 2048-bit keys are sufficient until 2030" [according to Wikipedia](#). An operating system running old software ought to be able to update its key size requirements, and programs using C++ networking may want to customize how they handle and accept key sizes.

§ 4.14. Cipher Suites

[boringssl](#) has deprecated some older cipher suites that they think are not being used any more. When a cipher suite has been proven insecure, it must be deprecated and its use must not be considered secure any more. In order to do this in a compatible way, the system must handle cipher suite selection and updating with the ability to customize behavior.

§ 4.15. SHA1 Deprecation

TLS1.2 specification says to use SHA1 because it was written well before SHA1 was [shown to not be secure](#). We need the ability to deprecate algorithms and protocols while maintaining source compatibility with simple software. Binary compatibility is probably out of the question, so we will probably need a special `std::networking` namespace that does not promise binary compatibility if we allow deep control of the algorithms used, but if we use a black-box things like this can be done underneath us.

§ 4.16. TLS1.0 Deprecation

[TLS 1.0 is definitely not secure](#), and TLS 1.1 had some updates in TLS 1.2. C++ implementations should support the latest and best practices in encryption. There needs to be a path to deprecation that can be done without breaking existing programs.

§ 4.17. Revoking Individual Certificates

The system might provide such a service. Sometimes people are desperately trying to broadcast that they have been breached. There should be a way to get updates, but maybe no requirements because sometimes you might not want to trust anyone to give you revoked certificate list updates. Online Certificate Status Protocol and Certificate Revocation Lists are used to distribute such revocations.

§ 4.18. Errors

There are a lot of errors. Many of them must result in failure or the entire protocol is insecure. We probably shouldn't ask the program if it wants to continue in such cases. Sometimes, though, we might want to continue. It's hard to decide where to draw the line sometimes.

§ 4.19. Default Ports

443 is the default port for HTTPS. Obviously the port must be configurable.

§ 4.20. Default Timeouts

Some network libraries use one minute of no response from a client or server as an indication that the connection has been lost, but some network conditions require waiting for more time for very important things. This must be configurable.

§ 4.21. Connection Caching

High level structures such as a cache of active connections can lead to great performance improvements but a client might want to terminate connections or have multiple connections to the same server for many reasons. Sometimes retrying a connection to the same server will result in fewer [hops](#). Such structures should not be included in the STL right now.

§ 4.22. HTTP Caching

HTTP and its caching is a box of worms we should probably leave to boost and other libraries for now. It's fraught with state, cookies, and other fun things.

§ 4.23. Extensions

TLS is designed to be quite extendable. There are some current extensions that we will want to support in a way that can be added to in the future. For example, in order to use the HTTP2 protocol, you must include an extension (ALPN) in the ClientHello TLS message indicating such support and you must verify the server sent a corresponding extension indicating it understands HTTP2. To support this, we should allow a property to be set with the ALPN information rather than having the C++ networking

user encode their own TLS extensions. Similar approaches should be taken with other TLS extensions in an expandable way.

This is not an exhaustive list of considerations, but with many of them at some point a line will need to be drawn and we will need to say "If you want to do that, implement your own library."

§ References

§ Informative References

[IETF-ARCHITECTURE]

[An Architecture for Transport Services](https://tools.ietf.org/html/draft-ietf-taps-arch-03). 2019-03-11. URL: <https://tools.ietf.org/html/draft-ietf-taps-arch-03>

[IETF-INTERFACE]

[An Abstract Application Layer Interface to Transport Services](https://tools.ietf.org/html/draft-ietf-taps-interface-03). 2019-03-11. URL: <https://tools.ietf.org/html/draft-ietf-taps-interface-03>

[N4771]

Jonathan Wakely. [Working Draft, C++ Extensions for Networking](https://wg21.link/n4771). 8 October 2018. URL: <https://wg21.link/n4771>

[P1861R0]

[Secure Connections in Networking TS](https://wg21.link/P1861R0). 2019-09-05. URL: <https://wg21.link/P1861R0>